

PROPUESTA DE TRABAJO DE FIN DE GRADO: SPHERE

Título	del	proyecto:	Proyecto	SPHERE.
Estudiante:		Saúl		Briceño
Grado:	Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)			
Tutor:		David		Oliván
Fecha:	25 noviembre de 2025			

PROPUESTA DE TRABAJO DE FIN DE GRADO: SPHERE	1
1. RESUMEN	2
Abstract	3
2. INTRODUCCIÓN.....	4
2.1. Contexto y justificación del tema	5
2.2. Relevancia académica y profesional	5
3. OBJETIVOS.....	6
3.1. Objetivo general	7
3.2. Objetivos específicos.....	7
4. MARCO TEÓRICO Y ESTADO DEL ARTE	8
4.1. Sistemas Multi-Agente (MAS) y LLMs	9
4.2. Arquitecturas Modernas Web	9
4.3. Inferencia Eficiente.....	9
4.4. Recuperación Aumentada (RAG).....	9
5. METODOLOGÍA	10
5.1. Diseño del proyecto y herramientas	10
A. Frontend e Interfaz Reactiva: React + Vite	10
B. Backend y API Asíncrona: FastAPI	10
C. Inteligencia Artificial y Optimización: Unsloth + PyTorch	11
D. Orquestación Agéntica: LangGraph.....	11
E. Persistencia Unificada: MongoDB Atlas	11
F. Containerización y Seguridad: Podman	11
G. Infraestructura de Despliegue Híbrido	12
5.2. Diseño Experimental e Hipótesis de Investigación.....	12
5.3. Definición de Recursos y Herramientas (Enfoque MCP)	12
5.4. Plan de trabajo y fases	13
6. RESULTADOS ESPERADOS.....	14
6.1. Posibles aplicaciones prácticas	15
7. VIABILIDAD ECONÓMICA Y MODELO DE NEGOCIO	16
7.1. Estructura de Costes (CAPEX y OPEX)	17
7.2. Modelo de Negocio y Segmentación de Clientes	18
7.3. Propuesta de Valor y Retorno (ROI) para el Cliente	20

8. CONCLUSIONES PRELIMINARES.....	21
8.1. Valor Añadido y Diferencial	22
8.2. Limitaciones y Líneas Futuras.....	22
9. BIBLIOGRAFÍA	22

1. RESUMEN

El presente Trabajo de Fin de Grado (TFG), titulado Proyecto SPHERE, propone el diseño, desarrollo y despliegue de una plataforma web full-stack que implementa una arquitectura multi-agente para la simulación de debates estratégicos empresariales. El sistema orquesta la interacción autónoma entre cuatro roles ejecutivos especializados (CEO, CTO, CMO, CFO), potenciados mediante una estrategia híbrida de Fine-Tuning (ajuste fino) y RAG (Generación Aumentada por Recuperación), y dotados de capacidad de ejecución técnica a través del estándar MCP (Model Context Protocol).

El proyecto abarca el ciclo de vida completo de desarrollo de software (SDLC) e integra una capa robusta de MLOps. Se han implementado pipelines de scraping automatizado para la recolección de conocimiento de dominio, generando datasets curados para ajustar modelos de lenguaje eficientes (SLMs como Llama 3) y validar experimentalmente la hipótesis de que modelos pequeños especializados pueden superar a modelos generalistas en tareas de nicho.

La arquitectura final despliega un patrón de infraestructura híbrida desacoplada: la orquestación de lógica de negocio y microservicios en entornos serverless de CPU (Google Cloud Run) y la inferencia intensiva delegada a clusters de GPU bajo demanda (Runpod Serverless). Esta configuración optimiza drásticamente la relación coste-latencia, demostrando la viabilidad técnica de sistemas de Inteligencia Artificial Agéntica de alto rendimiento en entornos de producción.

Abstract

This Bachelor's Thesis, titled Project SPHERE, proposes the design, development, and deployment of a full-stack web platform that implements a multi-agent architecture for simulating strategic business debates. The system orchestrates autonomous interactions among four specialized executive roles (CEO, CTO, CMO, CFO), enhanced through a hybrid strategy of **Fine-Tuning** and **RAG** (Retrieval-Augmented Generation), and empowered with technical execution capabilities via the **MCP (Model Context Protocol)** standard.

The project encompasses the complete Software Development Life Cycle (SDLC) and integrates a robust **MLOps** layer. Automated scraping pipelines have been implemented to gather domain-specific knowledge, creating curated datasets to fine-tune efficient language models (SLMs like Llama 3) and experimentally validate the hypothesis that specialized small models can outperform generalist models in niche tasks.

The final architecture deploys a decoupled hybrid infrastructure pattern: business logic orchestration and microservices on CPU serverless environments (**Google Cloud Run**) and intensive inference offloaded to on-demand GPU clusters (**Runpod Serverless**). This configuration drastically optimizes the cost-latency ratio, demonstrating the technical viability of high-performance Agentic AI systems in production environments.

2. INTRODUCCIÓN

2.1. Contexto y justificación del tema

En la actualidad, la Inteligencia Artificial ha evolucionado desde simples chatbots de respuesta única hacia sistemas agénticos capaces de mantener contextos complejos, simular roles específicos y ejecutar herramientas. Sin embargo, persiste una carencia de soluciones integradas y accesibles que permitan a las empresas visualizar cómo diferentes perfiles directivos interactúan y negocian entre sí ante un problema estratégico, al tiempo que generan entregables tangibles derivados de esa interacción.

Este proyecto se justifica al abordar el desafío crítico de la IA Agéntica: cerrar la brecha entre un modelo que "opina" y uno que "ejecuta" y "conoce". Para ello, se propone una arquitectura híbrida que combina la personalización de modelos con técnicas de Retrieval-Augmented Generation.

Esta combinación permite superar las limitaciones de conocimiento estático de los LLMs tradicionales. Al integrar arquitecturas modernas con la curación de datos propios y la conexión a bases documentales dinámicas, se busca transformar un modelo genérico en uno experto, capaz de responder con precisión, fundamentar sus argumentos en información actualizada y adaptarse a contextos estratégicos reales.

2.2. Relevancia académica y profesional

Este TFG demuestra la competencia del alumno en el diseño de arquitecturas distribuidas, la gestión de bases de datos NoSQL y la orquestación de contenedores seguros, simulando un entorno de producción real que enfrenta desafíos de latencia, gestión de estado mediante State Machines y optimización de costes bajo principios FinOps. Estas son habilidades clave en la ingeniería de software y DevOps contemporánea.

Asimismo, el proyecto integra competencias avanzadas de programación con habilidades emergentes de Ingeniería de IA. Se destaca la capacidad de crear datasets propios mediante scraping, ejecutar procesos de entrenamiento supervisado, implementar sistemas de recuperación semántica vectorial y desplegar herramientas funcionales mediante el protocolo MCP. Esta combinación diferencia el trabajo de una simple integración de APIs estándar, situándolo en la vanguardia del desarrollo de Sistemas Cognitivos, donde la personalización del modelo y el acceso a datos en tiempo real son esenciales para la toma de decisiones empresariales.

3. OBJETIVOS

3.1. Objetivo general

Construir, desplegar y validar una plataforma full-stack robusta que simule una Junta Directiva Virtual conformada por agentes de IA especializados. El sistema debe permitir no solo el debate estratégico entre perfiles directivos, sino también la generación automática de entregables preliminares mediante el uso de herramientas conectadas, garantizando una experiencia de usuario fluida, una arquitectura mantenible y económicamente viable.

Estos agentes estarán entrenados con datasets creados ad-hoc para especializarlos en sus respectivas áreas —Dirección General, Tecnología, Marketing y Finanzas—, aportando realismo y profundidad a las dinámicas de interacción simuladas. Además, se integrará Retrieval-Augmented Generation para que los agentes puedan consultar bases documentales actualizadas y fundamentar sus decisiones en información reciente y relevante.

3.2. Objetivos específicos

Para alcanzar el objetivo general, se establecen los siguientes hitos:

- **Ingeniería de Datos (ETL):**

Desarrollar pipelines de scraping ético para extraer corpus de texto de fuentes relevantes (blogs tecnológicos para el CTO, noticias financieras para el CFO, papers académicos para el CEO, informes de mercado para el CMO).

Limpiar, normalizar y estructurar la información en formatos aptos para entrenamiento (JSONL).

- **Fine-Tuning y Especialización:**

Ajustar modelos Open Source (Llama 3 / Mistral) mediante técnicas de optimización de recursos como QLoRA, adaptando tono y conocimiento técnico a cada rol directivo.

- **Integración de RAG:**

Implementar un sistema de Retrieval-Augmented Generation que permita a los agentes consultar bases documentales vectorizadas y enriquecer sus respuestas con información actualizada y contextual.

Evaluar cómo la combinación de fine-tuning + RAG mejora el realismo y la relevancia de los debates estratégicos.

- **Arquitectura de Backend e Inferencia:**

Diseñar un backend asíncrono con FastAPI que orqueste la comunicación con un servicio de inferencia desacoplado (vLLM o TGI en Runpod), garantizando latencias de respuesta (TTFT) inferiores a 2 segundos.

- **Implementación de Herramientas MCP:**

Dotar a los agentes de capacidad de ejecución mediante servidores MCP básicos (ej. creación de repositorios en GitHub, generación de PDFs, ejecución de scripts).

- **Interfaz de Usuario y Persistencia:**

Desarrollar una SPA en React que gestione el streaming de tokens en tiempo real y visualice el estado de las herramientas agénticas.

Integrar persistencia en MongoDB para almacenar historiales de chat y configuraciones de simulación.

- **Despliegue y CI/CD:**

Configurar pipelines de integración continua y desplegar la solución en un entorno de nube híbrida (GCP + Runpod), asegurando escalabilidad y mantenibilidad.

- **Validación Experimental:**

Evaluar la coherencia de los roles, la eficiencia de costes frente a modelos propietarios y el impacto del uso de RAG en la mejora de realismo y sostenibilidad económica.

4. MARCO TEÓRICO Y ESTADO DEL ARTE

4.1. Sistemas Multi-Agente (MAS) y LLMs

El proyecto se fundamenta en la teoría de Sistemas Multi-Agente (MAS), donde agentes autónomos colaboran o compiten para resolver problemas complejos. Se explora cómo la especialización mediante datos propios mejora la calidad del debate frente al simple prompt engineering.

Con la llegada de los Large Language Models (LLMs) como GPT-4 o Llama 3, es posible dotar a estos agentes de un razonamiento semántico avanzado. Se incorporan técnicas como Chain of Thought (CoT) y Role-Playing para definir las personalidades de los directivos, enriqueciendo la dinámica de interacción y evolucionando hacia sistemas capaces de usar herramientas (Tool Use) para interactuar con el mundo exterior.

4.2. Arquitecturas Modernas Web

El proyecto revisará las técnicas modernas de adaptación de modelos, incluyendo el scraping mediante herramientas como BeautifulSoup o Selenium para la extracción de datos no estructurados, y el uso de PEFT (Parameter-Efficient Fine-Tuning), que permite entrenar modelos de gran escala en hardware de consumo o en nubes económicas ajustando solo una fracción de sus parámetros mediante enfoques como LoRA.

Asimismo, se analizará el estado del arte en desarrollo web, contrastando arquitecturas monolíticas frente a microservicios, adoptando una postura moderna basada en el uso de contenedores seguros (Podman) para garantizar la paridad entre entornos de desarrollo y producción. Se integrarán patrones de diseño como el Repository Pattern en el backend para desacoplar la lógica de negocio de la base de datos, junto con enfoques asíncronos que favorecen la escalabilidad y mantenibilidad.

4.3. Inferencia Eficiente

El proyecto analizará las tecnologías actuales para el servicio de modelos de IA, priorizando alternativas a las APIs propietarias costosas mediante el uso de librerías como vLLM y servidores de inferencia optimizados. Estos motores emplean algoritmos como PagedAttention para optimizar la memoria de la GPU, permitiendo servir modelos Open Source con un rendimiento comparable al de soluciones comerciales.

Asimismo, se abordará la integración de estas cargas de trabajo intensivas de inferencia con arquitecturas web modernas, ligeras y asíncronas, empleando FastAPI junto con mecanismos de comunicación en tiempo real como WebSockets o Server-Sent Events (SSE), garantizando eficiencia, escalabilidad y una experiencia de usuario fluida.

4.4. Recuperación Aumentada (RAG)

Para enriquecer la capacidad de los agentes y evitar respuestas genéricas, se integrará un módulo de Retrieval-Augmented Generation (RAG). Este enfoque combina la generación de texto de los LLMs con la recuperación de información desde una base documental actualizada (MongoDB, índices vectoriales con FAISS o Weaviate).

El uso de RAG permitirá:

- Acceso a conocimiento dinámico: Los agentes podrán consultar fuentes externas (papers, informes financieros, blogs técnicos) en tiempo real.
- Mayor realismo en los debates: Cada rol directivo podrá fundamentar sus argumentos en datos recientes y relevantes.
- Optimización de costes: Al reducir la necesidad de entrenamientos extensivos, se aprovecha la recuperación contextual para mantener precisión sin incrementar el gasto computacional.
- Escenarios híbridos: Combinación de fine-tuning para especialización de tono con RAG para actualización constante del conocimiento.

5. METODOLOGÍA

5.1. Diseño del proyecto y herramientas

La arquitectura del proyecto se ha diseñado seleccionando herramientas que maximizan la eficiencia, la seguridad y la experiencia de usuario en tiempo real.

A. Frontend e Interfaz Reactiva: React + Vite

- Selección: Biblioteca React inicializada con Vite para un entorno de desarrollo ultrarrápido.
- Justificación Técnica: La naturaleza de la IA generativa requiere una gestión de estado compleja en el cliente para renderizar la transmisión de texto (token streaming) sin recargar la página. El Virtual DOM de React permite actualizar la interfaz miles de veces por minuto de forma eficiente, algo crítico para simular la "velocidad de pensamiento" de los agentes. Se utiliza Tailwind CSS para un diseño de componentes rápido y mantenible.

B. Backend y API Asíncrona: FastAPI

- Selección: Framework FastAPI (Python).
- Justificación Técnica: Elegido por su soporte nativo de concurrencia (`async/await`). A diferencia de frameworks síncronos como Django o Flask, FastAPI puede gestionar cientos de conexiones long-polling o SSE (Server-Sent Events) simultáneas mientras espera la respuesta de la GPU, evitando cuellos de botella en la orquestación del debate. Además, la validación de datos con Pydantic asegura que los inputs hacia los agentes cumplan estrictamente los esquemas requeridos.

C. Inteligencia Artificial y Optimización: Unsloth + PyTorch

- Selección: Ecosistema Hugging Face optimizado con Unsloth.
- Justificación Técnica: Unsloth permite realizar Fine-Tuning (QLoRA) de modelos Llama-3 reduciendo el consumo de VRAM en un 60% y acelerando el entrenamiento 2x. Esto democratiza el acceso a modelos expertos, permitiendo entrenar y ejecutar la inferencia en GPUs de consumo (NVIDIA L4) en lugar de costosas A100, clave para la viabilidad económica del proyecto.

D. Orquestación Agéntica: LangGraph

- Selección: LangGraph (sobre LangChain).
- Justificación Técnica: Se descarta el uso de cadenas lineales simples en favor de una arquitectura basada en Grafos de Estado (Stateful Graphs). Esto permite definir flujos cíclicos donde los agentes pueden "reflexionar", corregirse a sí mismos o solicitar feedback humano antes de avanzar, capacidades esenciales para una simulación estratégica compleja.

E. Persistencia Unificada: MongoDB Atlas

- Selección: MongoDB con capacidad Vector Search.
- Justificación Técnica: Consolida en un solo servicio la base de datos documental (historial de chats, usuarios) y el índice vectorial (embeddings para RAG), simplificando la infraestructura y reduciendo la latencia de red.

F. Containerización y Seguridad: Podman

- Selección: Podman como motor de contenedores (OCI Compliant).
- Justificación Técnica (Diferencial): A diferencia de Docker, Podman utiliza una arquitectura Daemonless (sin demonio central) y Rootless (sin privilegios de administrador por defecto). Esto reduce drásticamente la superficie de ataque en el entorno de producción. Al ser compatible con los estándares OCI, permite utilizar las mismas imágenes Docker, pero con una capa de seguridad superior, demostrando un enfoque Security-First desde el diseño.

G. Infraestructura de Despliegue Híbrido

- Orquestación (CPU): Google Cloud Run. Aloja el Frontend y Backend. Escala a cero cuando no hay uso, minimizando costes.
- Inferencia (GPU): Runpod Serverless. Ejecuta los contenedores de IA bajo demanda. Se elige por ofrecer una relación coste-rendimiento superior a los proveedores hyperscalers (AWS/Google) para cargas de trabajo de GPU puntuales.

5.2. Diseño Experimental e Hipótesis de Investigación

- **H1 - Hipótesis de Especialización:** Un modelo compacto (8B parámetros) con Fine-Tuning (QLoRA) sobre datos de dominio superará en coherencia de rol a un modelo generalista (70B+) instruido solo con Prompt Engineering.
- **H2 - Hipótesis de Latencia Híbrida:** La arquitectura desacoplada permitirá mantener un Time-To-First-Token (TTFT) promedio inferior a 2 segundos en condiciones de arranque en caliente (warm start), validando la viabilidad de sistemas interactivos en tiempo real utilizando infraestructura serverless bajo demanda, optimizando el coste por token frente a instancias dedicadas.
- **H3 - Hipótesis de Recuperación Aumentada (RAG):** La integración de RAG mejorará la relevancia y realismo de los debates estratégicos, al permitir que los agentes fundamenten sus respuestas en información actualizada y específica, reduciendo la dependencia de entrenamientos extensivos y optimizando costes.

5.3. Definición de Recursos y Herramientas (Enfoque MCP)

Se implementará el estándar Model Context Protocol (MCP) para desacoplar las capacidades de los agentes. Cada rol consumirá recursos a través de conectores estandarizados (MCP Servers):

- **CEO – Orchestrator.Delegate (LangGraph):** Máquina de estados que asigna turnos y tareas, con acceso a RAG para fundamentar decisiones estratégicas.
- **CMO – Content.Draft (FileSystem):** Genera estrategias de copy en Markdown persistente, apoyándose en RAG para incorporar tendencias de mercado recientes.
- **CTO (MCP GitHub Server):** Interfaz estandarizada para operaciones `git init`, gestión de ramas y lectura de repositorios.
- **CFO (MCP Reporting Server):** Microservicio encapsulado para la generación de PDFs con ReportLab y consultas financieras.

5.4. Plan de trabajo y fases

El desarrollo se estructura en las siguientes fases:

Fase 1: Diseño, Descubrimiento y Arquitectura.

Definición de roles y fuentes de datos objetivo, junto con los requisitos funcionales (roles, chat, persistencia) y no funcionales (latencia < 2s). Configuración inicial del monorepo y diseño de la arquitectura general.

Fase 2: Ingeniería de Datos y Prototipado del Backend.

Desarrollo de spiders para el scraping ético de texto, limpieza y formateo del Dataset-V1. Implementación inicial de la lógica de orquestación de turnos y conexión con la base de datos.

Fase 3: Entrenamiento e Integración de IA.

Ejecución de entrenamientos con QLoRA, evaluación de la pérdida (loss) y pruebas cualitativas de los modelos resultantes. Configuración del servicio de

inferencia y diseño de prompts específicos para cada rol directivo (CEO, CTO, CMO, CFO).

Implementación de pipeline de evaluación automática usando LLM-as-a-Judge (usando un modelo potente como juez, ej. Gemini 3) para puntuar la coherencia del rol y la calidad técnica de las respuestas (Score 1-5)

Fase 4: Desarrollo Backend/Inferencia y Frontend.

Creación de la API que sirve los modelos y orquesta los turnos, integrando el servicio de inferencia. Construcción de la interfaz de usuario en React para visualizar el debate y paneles de control asociados.

Fase 5: Pruebas y Optimización.

Realización de pruebas y ajustes de hiperparámetros de los modelos y optimización de la infraestructura para garantizar rendimiento y latencia adecuada.

Fase 6: Despliegue y Documentación.

Puesta en producción de la solución, despliegue en entorno de nube híbrida y redacción de la memoria final del proyecto.

6. RESULTADOS ESPERADOS

Al finalizar el proyecto, se espera obtener:

- **Cuatro Modelos Expertos:** Adaptadores LoRA específicos para CEO, CTO, CMO y CFO, capaces de demostrar vocabulario, prioridades y estilos argumentativos distintos, manteniendo coherencia en sus intervenciones a lo largo de la conversación.
- **Dataset Propio:** Conjunto de datos curado, limpiado y estructurado en formatos aptos para entrenamiento (JSONL), valioso como recurso reutilizable en futuros proyectos.
- **Plataforma Web Funcional:** Aplicación accesible vía navegador que permita simular debates empresariales entre agentes especializados, con experiencia fluida y dinámica, incluyendo generación automática de entregables (informes, repositorios, planes de marketing).
- **Pipeline de MLOps:** Documentación completa del proceso, desde la extracción del dato crudo hasta el despliegue del modelo, asegurando trazabilidad y reproducibilidad.
- **Repositorio de Código:** Codebase limpio, documentado y testado, siguiendo buenas prácticas de ingeniería de software (Clean Code, Clean Architecture, SOLID), que facilite colaboración y mantenimiento.
- **Infraestructura como Código (IaC):** Scripts y configuraciones reproducibles para despliegue en contenedores y replicables en cualquier proveedor cloud, garantizando portabilidad y escalabilidad.

6.1. Posibles aplicaciones prácticas

La plataforma posee un amplio potencial de aplicación en diversos contextos:

- **EdTech (MBA Virtual):** Herramienta educativa para estudiantes de administración y negocios, facilitando la simulación de escenarios de crisis, negociación y toma de decisiones estratégicas en entornos controlados.

- **SaaS para Startups:** Una “consultora de bolsillo” que permite a emprendedores validar ideas técnicas o financieras rápidamente antes de contratar personal humano, acelerando procesos creativos y de planificación.
- **Consultoría Empresarial:** Apoyo a empresas en procesos de planificación estratégica, benchmarking y análisis de escenarios, ofreciendo simulaciones rápidas y fundamentadas que complementan el trabajo de consultores humanos y reducen costes iniciales.
- **Automatización de Productividad (Agentic Workflow):** Solución orientada a freelancers, desarrolladores y creadores independientes que requieren una fuerza de trabajo digital bajo demanda. En este contexto, la plataforma se utiliza no para el debate, sino para la ejecución de tareas operativas a través del protocolo MCP (ej. inicialización de repositorios de código por el Agente CTO o generación de facturas proforma por el Agente CFO), actuando como un equipo de asistentes especializados para aumentar la productividad individual.

7. VIABILIDAD ECONÓMICA Y MODELO DE NEGOCIO

7.1. Estructura de Costes (CAPEX y OPEX)

A. CAPEX

Se ha valorado el desarrollo utilizando tarifas de mercado para un perfil de Arquitectura de Soluciones de IA, dada la especialización requerida en MLOps, cuantización de modelos y orquestación distribuida, tareas que exceden significativamente el alcance de un desarrollo web convencional.

- Perfil: Junior AI Solutions Architect / ML Engineer.
- Tarifa de mercado: 45 €/hora.
- Esfuerzo estimado: 300 horas.
- Valoración Total: 13.500 €.

Nota: Este coste se considera una inversión "in-kind" (en especie) en el contexto académico, representando el valor real de mercado del activo tecnológico desarrollado.

B. OPEX

Para validar la escalabilidad del modelo de negocio, se han calculado los costes asumiendo un entorno de producción real bajo condiciones de carga intensiva (High-Density Output), donde los agentes generan entregables extensos (código, informes) en cada turno. Asimismo, se ha prescindido de las capas gratuitas para garantizar SLAs y redundancia, estructurando los costes en dos categorías: fijos de infraestructura y variables por uso.

1. Costes Variables:

Inferencia (Compute – Runpod Serverless)

- Recurso: NVIDIA L4 (24GB VRAM)
- Coste unitario: 0,39 €/hora (facturación por segundo)
- Métrica de uso: 5 minutos promedio por sesión.

Cálculo:

$$(5 \text{ min} / 60) \times 0,39 \text{ €} \approx 0,033 \text{ €}$$

Orquestación (Google Cloud Run)

- Coste unitario: 0,000024 € vCPU/segundo

Cálculo:

$$300 \text{ segundos} \times 0,000024 \text{ €} \approx 0,007$$

Coste marginal total por simulación:

$$0,033 \text{ € (GPU)} + 0,007 \text{ € (CPU)} = 0,040 \text{ €}$$

2. Costes Fijos de Infraestructura (Base Operativa)

- Almacenamiento Persistente (GPU Volume Cache): 2,00 €/mes.
- Base de Datos Producción (MongoDB Atlas - Tier M10): 15,00 €/mes.
- DevOps & Registry (GCR/Build): 3,00 €/mes.
- Total Costes Fijos: 20,00 €/mes.

3. Resumen Financiero Mensual

- **Costes Fijos (Infraestructura):** 20,00 €
- **Costes Variables (500 simulaciones completas):** 20,00 €
- **TOTAL OPEX MENSUAL: 40,00 €**

Punto de Equilibrio (Break-even)

Dado que los costes fijos son extremadamente bajos y el coste variable es de céntimos, el sistema alcanza el punto de equilibrio con un solo usuario suscrito al plan PRO (29 €/mes). Esto valida la hipótesis de la Arquitectura Híbrida Desacoplada: máxima rentabilidad y riesgo financiero mínimo desde el primer cliente.

7.2. Modelo de Negocio y Segmentación de Clientes

SPHERE opera bajo un modelo SaaS B2B/B2C híbrido, atacando dos segmentos de mercado diferenciados con una misma plataforma tecnológica:

1. **Segmento Corporativo (B2B):** Directivos y Managers que buscan simular escenarios de riesgo y toma de decisiones estratégicas ("Risk-Free Strategy Simulation").
2. **Segmento "Prosumer" / Freelance (B2C):** Desarrolladores y emprendedores que utilizan la capacidad de ejecución de los agentes (protocolo MCP) para automatizar tareas técnicas y administrativas ("AI Workforce as a Service").

Esquema de Precios (Tiered Pricing):

- **FREE (Discovery):**
 - o Acceso limitado a 3 debates/mes (solo lectura).
 - o Objetivo: *Product-Led Growth* (PLG). Demostración de capacidad técnica para captación masiva de leads.
- **PRO (Standard - 29€/mes):**
 - o *Enfoque:* Para Managers y "Solo-preneurs".
 - o **Simulación:** Debates ilimitados para análisis de estrategia.
 - o **Ejecución (Utility):** Acceso directo a las herramientas MCP. El usuario puede pedirle al "Agente CTO" que genere la estructura de un proyecto real (código descargable) o al "Agente CFO" que audite un CSV subido por el usuario.
 - o **Valor:** Sustituye tareas repetitivas de un asistente junior por una fracción del coste.
- **ENTERPRISE (Custom - Desde 499€/mes + Setup):**

- o *Enfoque:* Grandes Corporaciones.
- o **Digital Twin:** Fine-tuning con datos históricos privados de la empresa.
- o **Seguridad:** Despliegue en VPC (Virtual Private Cloud) o *On-Premise* para garantizar que ningún dato confidencial salga de su infraestructura.
- o Integración con ERP/CRM corporativos.

7.3. Propuesta de Valor y Retorno (ROI) para el Cliente

La propuesta de valor de SPHERE se articula diferenciando las necesidades críticas de dos segmentos, traduciendo las capacidades técnicas (IA Agéntica) en métricas de rentabilidad tangibles.

A. Para la Empresa (B2B): El "Virtual War Room"

Problema: Una reunión estratégica de 2 horas entre cuatro directivos reales (CEO, CTO, CMO, CFO) tiene un coste hundido estimado superior a 1.000 €, sin garantizar conclusiones accionables.

Solución SPHERE: Ofrece un entorno de simulación predictiva (Sandbox) donde se pueden iterar decenas de escenarios en minutos antes de convocar a los humanos.

ROI (Retorno): Se mide en la validación de hipótesis a coste marginal cero. Detectar una inconsistencia técnica (CTO) o financiera (CFO) en la fase de simulación evitar la toma de decisiones fallidas, cuyo coste de rectificación en producción sería exponencialmente mayor (Shift-Left Testing aplicado a la estrategia de negocio).

B. Para el Profesional Independiente (B2C): "Fractional C-Suite as a Service"

Problema: Freelancers, emprendedores y solopreneurs carecen de presupuesto para contratar consultoría especializada o delegar tareas técnicas/administrativas.

Solución SPHERE: Democratiza el acceso a un equipo directivo y técnico mediante la Capacidad Agéntica Aumentada. Gracias al protocolo MCP, el usuario no solo recibe consejos, sino trabajo ejecutado (código, informes, planes).

ROI (Retorno):

Comparativa: Contratar a un gestor o desarrollador junior cuesta 25€/hora. La suscripción mensual PRO (29€) equivale al coste de una sola hora de consultoría

humana, ofreciendo disponibilidad 24/7 de cuatro roles expertos. El retorno es inmediato al automatizar la primera tarea compleja.

8. CONCLUSIONES PRELIMINARES

8.1. Valor Añadido y Diferencial

El Proyecto SPHERE trasciende el concepto tradicional de aplicación web para adentrarse en la Ingeniería de Sistemas Agénticos, una disciplina emergente con alta demanda. Su principal valor reside en la democratización del acceso a herramientas de estrategia empresarial mediante el uso de Inteligencia Artificial Eficiente. Al demostrar que modelos Open Source de 8B parámetros, correctamente ajustados y orquestados en arquitecturas serverless, pueden competir en utilidad con grandes modelos propietarios, el proyecto valida un nuevo paradigma de desarrollo de software: más económico, privado y personalizable.

8.2. Limitaciones y Líneas Futuras

Si bien la arquitectura propuesta optimiza la latencia, se asume que la calidad del razonamiento en escenarios extremadamente complejos dependerá de la calidad del dataset curado (Garbage In, Garbage Out). Como líneas futuras, se contempla la implementación de Reinforcement Learning from Human Feedback (RLHF) para refinar el comportamiento de los agentes basándose en la retroalimentación de los usuarios PRO, así como la exploración de NPU (Neural Processing Units) en el borde (Edge Computing) para reducir aún más la dependencia de la nube.

En conclusión, SPHERE no solo es viable técnicamente, sino que presenta un modelo de negocio sostenible desde el primer usuario, cerrando la brecha entre la investigación académica en IA y las necesidades reales del tejido empresarial.

9. BIBLIOGRAFÍA

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, W., Wang, L., Wang, Y., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685.
- Mitchell, R. (2018). *Web Scraping with Python* (2ª ed.). O'Reilly Media.
- Hugging Face. (2024). *TRL and PEFT libraries*. Recuperado de <https://huggingface.co/docs>
- Lubanovic, B. (2023). *FastAPI: Modern Python Web Development*. O'Reilly Media.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30.
- React Team. (2024). *React Documentation*. Recuperado de <https://react.dev/>
- Kwon, W., Lee, S., Lee, J., Kim, J., Kim, Y., & Han, D. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Proceedings of the ACM Symposium on Operating Systems Principles (SOSP '23).
- Fowler, M., & Lewis, J. (2014). *Microservices: a definition of this new architectural style*. Recuperado de <https://martinfowler.com/articles/microservices.html>